



**COMSATS University Islamabad,  
Abbottabad Campus**

**SOFTWARE DESIGN  
DESCRIPTION  
(SDD DOCUMENT)**

**for**

**RomaSub.AI:  
Roman Urdu Captions Generator**

***By***

|                              |                              |
|------------------------------|------------------------------|
| <b>Muttayyab Abdurrehman</b> | <b>CIIT/FA22-BSE-046/ATD</b> |
| <b>Muhammad Hashir</b>       | <b>CIIT/FA22-BSE-031/ATD</b> |
| <b>Muneeb Khan</b>           | <b>CIIT/FA22-BSE-032/ATD</b> |

***Supervisor***

**Dr. Osman Khalid**

***Bachelor of Science in Software Engineering (2022-2026)***

# Table of Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1. INTRODUCTION:</b>                                  | <b>7</b>  |
| <b>2. DESIGN METHODOLOGY AND SOFTWARE PROCESS MODEL:</b> | <b>7</b>  |
| <b>3. SYSTEM OVERVIEW:</b>                               | <b>7</b>  |
| 3.1 ARCHITECTURAL DESIGN:                                | 8         |
| 3.2 ARCHITECTURE DIAGRAM:                                | 8         |
| 3.3 ACTIVITY DIAGRAM:                                    | 10        |
| <b>4. DESIGN MODELS:</b>                                 | <b>11</b> |
| 4.1 DATA FLOW DIAGRAM:                                   | 11        |
| 4.2 STATE TRANSITION DIAGRAM:                            | 12        |
| <b>5. DATA DESIGN:</b>                                   | <b>13</b> |
| 5.1 MAJOR DATA ENTITIES & STRUCTURE                      | 13        |
| 5.2 DATABASE / STORAGE SOLUTIONS                         | 14        |
| 5.3 DATA DICTIONARY (DETAILED TABLES)                    | 15        |
| <b>6. ALGORITHM AND IMPLEMENTATION:</b>                  | <b>17</b> |
| 6.1 ASR PROCESSING ALGORITHM                             | 17        |
| 6.2 TRANSLITERATION ALGORITHM                            | 18        |
| 6.3 SUBTITLE EXPORT ALGORITHM                            | 18        |
| <b>7. SOFTWARE REQUIREMENTS TRACEABILITY MATRIX:</b>     | <b>18</b> |
| <b>8. INTERFACE DESIGN:</b>                              | <b>19</b> |
| 8.1 SCREEN IMAGES:                                       | 19        |
| 8.2 SCREEN OBJECTS AND ACTIONS:                          | 21        |
| <b>9 APPENDIX I</b>                                      | <b>22</b> |

## List of Tables

|                                    |      |
|------------------------------------|------|
| TABLE 5.1 DD-USER .....            | 15   |
| TABLE 5.2 DD-SUBTITLE .....        | 15   |
| TABLE 5.3 DD-PASSWORDRESET .....   | 1516 |
| TABLE 5.4 DD-USERFEEDBACK .....    | 17   |
| TABLE 7.1 TRACEABILITY MATRIX..... | 18   |

## List of Figures

|                                          |    |
|------------------------------------------|----|
| FIGURE 3.1 ARCHITECTURE DIAGRAM.....     | 9  |
| FIGURE 3.2 ACTIVITY DIAGRAM .....        | 10 |
| FIGURE 4.1.1 DFD LEVEL 0 .....           | 11 |
| FIGURE 4.1.2 DFD LEVEL 1 .....           | 11 |
| FIGURE 4.2 STATE TRANSITION DIAGRAM..... | 12 |
| FIGURE 8.1 REGISTER PAGE .....           | 20 |
| FIGURE 8.2 LOGIN PAGE .....              | 20 |
| FIGURE 8.3 DASHBOARD.....                | 20 |

## Revision History

| Name | Date | Reason for changes | Version |
|------|------|--------------------|---------|
|      |      |                    |         |
|      |      |                    |         |

## Application Evaluation History

| Comments (by committee) | Action Taken |
|-------------------------|--------------|
|                         |              |
|                         |              |

**Supervised By**  
**Dr. Osman Khalid**

Signature\_\_\_\_\_

## 1. Introduction:

RomaSub.AI is a desktop application designed to automatically generate Roman Urdu subtitles from Urdu speech in audio and video files. The system addresses a critical accessibility gap for over 50 million Urdu speakers who use Roman Urdu (Latin script) for digital communication.

The project scope covers the following modules: User Authentication, Video/Audio Input Handler, Automated Speech Recognition (ASR), Dataset Generation for Fine-Tuning, Fine Tuning and Training, Transliteration Engine, Real-Time Subtitle Display, Subtitle Editor, Export and File Management, and Feedback and Learning System.

The system uses OpenAI Whisper for Urdu speech recognition and a Transformer-based M2M100 model for transliteration. It is built as a Flutter desktop application with a FastAPI backend.

## 2. Design Methodology and Software Process Model:

The design of RomaSub.AI follows a Structural (Procedural) software development methodology. This approach is selected because the system operates through a sequence of well-defined functional stages, such as User Authentication, Video Input Handling, Automated Speech Recognition (ASR), Transliteration Pipeline, Subtitle Generation, and Export Module, each of which can be implemented as an independent procedure with a clear role in the processing workflow. Procedural principles such as functional decomposition, modular sequencing, and linear data flow support a clean and highly manageable architecture for both the Flutter-based frontend and the Python backend, ensuring that the system remains predictable, easy to debug, and simple to optimize. By structuring the system as a pipeline—where each stage transforms the input before passing it to the next—the methodology enables efficient execution, minimizes unnecessary complexity, and allows individual modules (such as ASR, transliteration, or subtitle formatting) to be updated or replaced without affecting the overall design.

RomaSub.AI is developed using an Iterative and Incremental Process Model. This model provides structured development cycles, continuous user feedback, and the flexibility to incorporate improvements throughout the project lifecycle. Because the system integrates AI models that require ongoing fine-tuning, the iterative approach allows the team to first develop core features—such as video uploading and ASR transcription—and then enhance accuracy and functionality—such as subtitle editing and feedback-driven model improvements—in subsequent increments. This ensures that the final solution meets user needs, system requirements, and performance expectations effectively.

## 3. System Overview:

RomaSub.AI is a complete subtitle generation system that converts Urdu speech into Roman Urdu text. Users can upload audio or video files, and the system automatically transcribes the speech,

converts it to Roman Urdu, and displays synchronized subtitles. The system also allows editing and exporting subtitles in standard formats.

### **3.1 Architectural Design:**

The system is built using a layered architecture that separates the user interface, processing logic, and data management. The Flutter desktop application functions as the client layer, allowing users to upload files, play videos, edit subtitles, and export results. All client requests are handled by the FastAPI-based API layer, which routes them to the service layer responsible for authentication, media processing, ASR operations, and transliteration logic. The AI/ML layer integrates OpenAI Whisper for speech recognition and the M2M100 model for converting Urdu text into Roman Urdu. Finally, the PostgreSQL-powered data layer securely stores user information, feedback, and project metadata to support smooth and reliable system operations.

#### **3.1.1 Client Layer:**

The Flutter desktop application serves as the primary user interface. Users can upload media files, view videos, edit subtitles, and export final outputs seamlessly.

#### **3.1.2 API Layer:**

The FastAPI backend exposes structured REST endpoints. It receives client requests and delegates them to the relevant service modules for processing.

#### **3.1.3 Service Layer:**

This layer contains all core business logic, including authentication, media handling, ASR orchestration, and transliteration workflows. It ensures clean separation between API routing and computational operations.

#### **3.1.4 AI/ML Layer:**

OpenAI Whisper performs high-quality speech recognition on uploaded audio/video. The M2M100 model converts Urdu text into Roman Urdu, enabling accurate and automated transliteration.

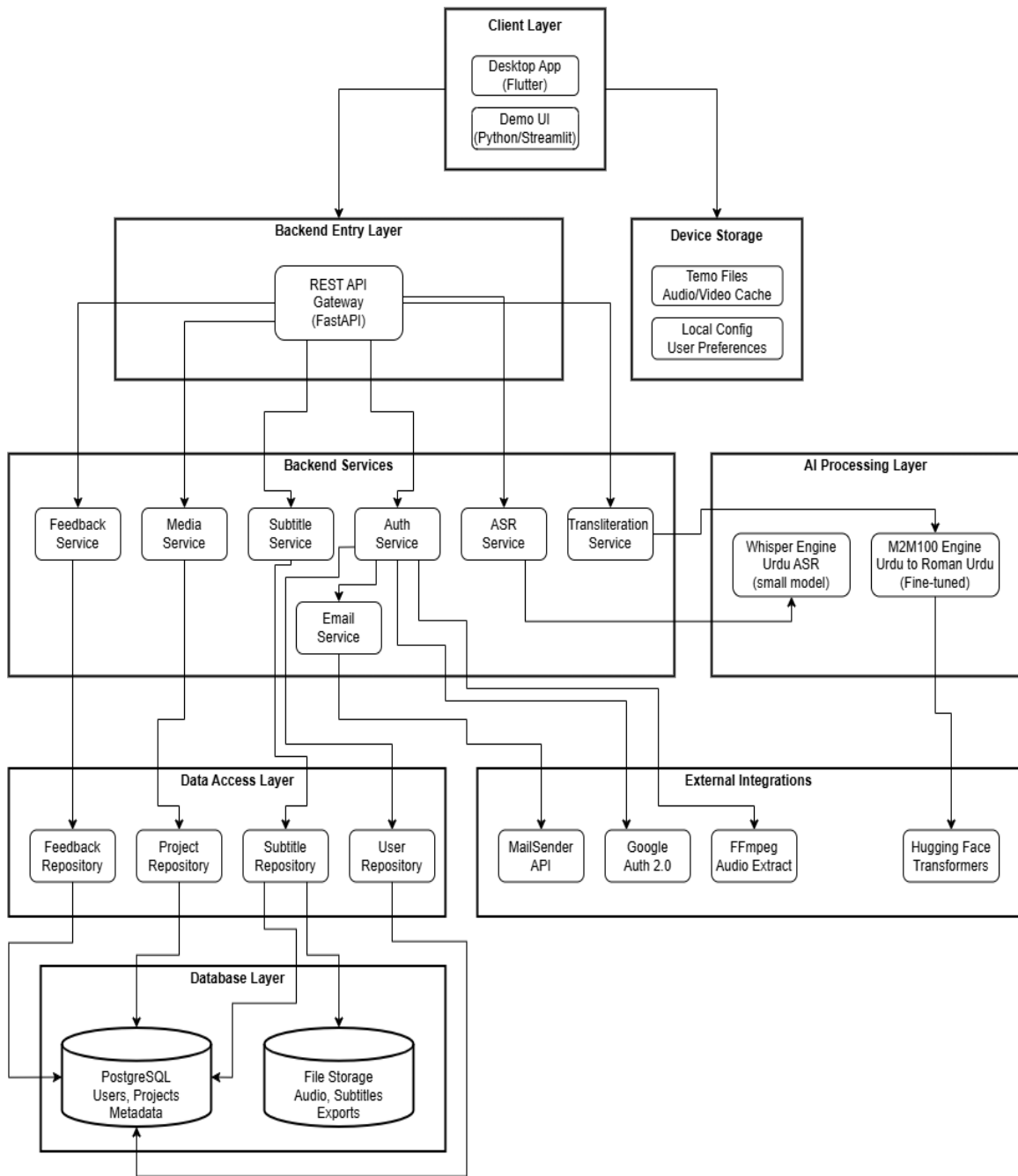
#### **3.1.5 Data Layer:**

A PostgreSQL database stores user profiles, project metadata, and feedback logs. It ensures data consistency, secure storage, and efficient retrieval for system operations.

### **3.2. Architecture Diagram:**

This Figure 3.1 presents architectural diagram illustrating the major components of RomaSub.AI and their interactions, providing a clear overview of the system's structure.

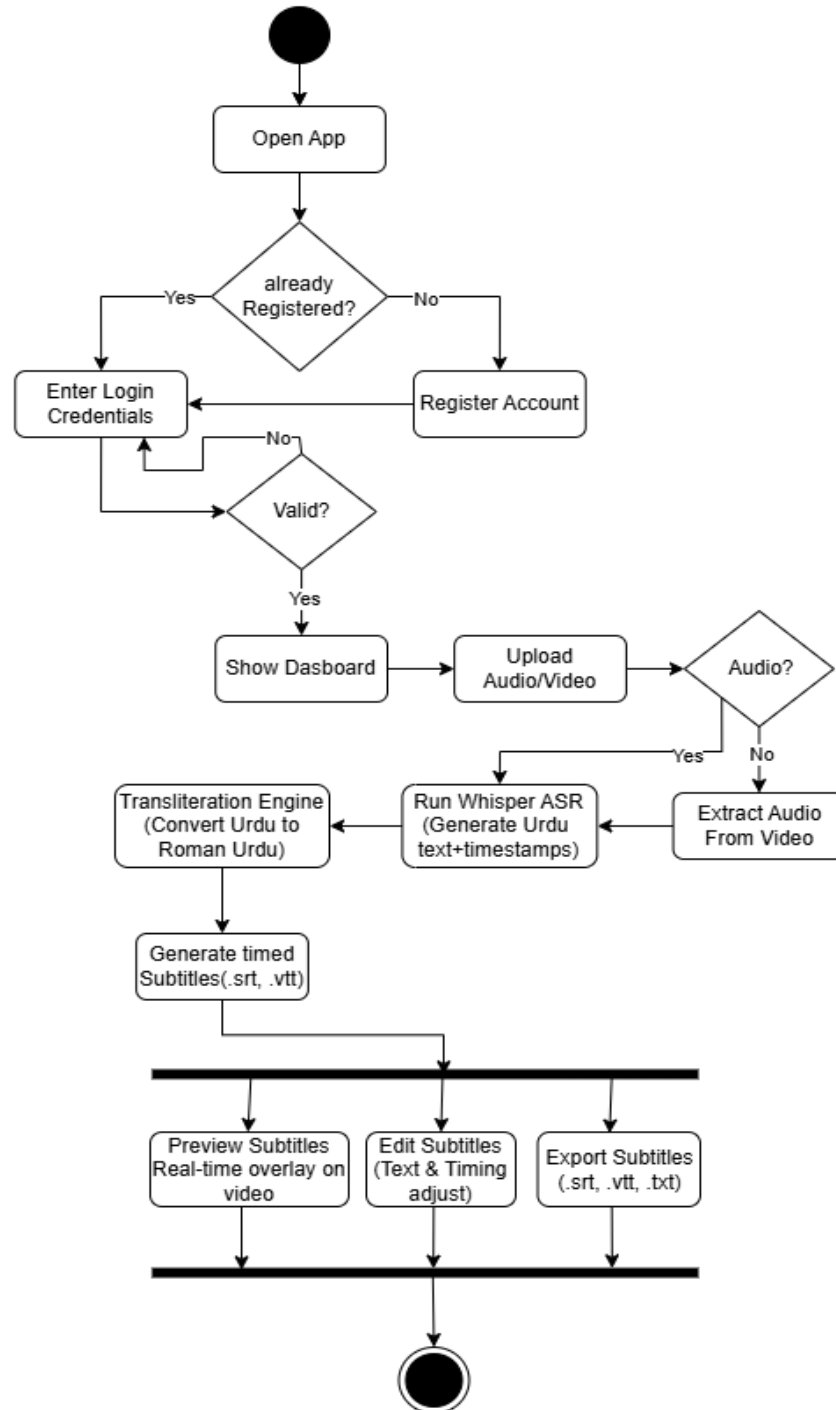




**Figure 3.1** Architecture Diagram

### 3.3. Activity Diagram:

This Figure 3.2 activity diagram depicting the flow of major processes within the RomaSub.AI.



**Figure 3.2** Activity Diagram

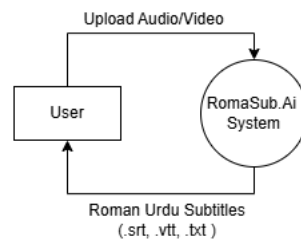
## 4. Design Models:

This section presents the key design models including dataflow and sequence and state transition diagrams that illustrate the structure and dynamic behaviour of the RomaSub.AI.

### 4.1. Data Flow Diagram:

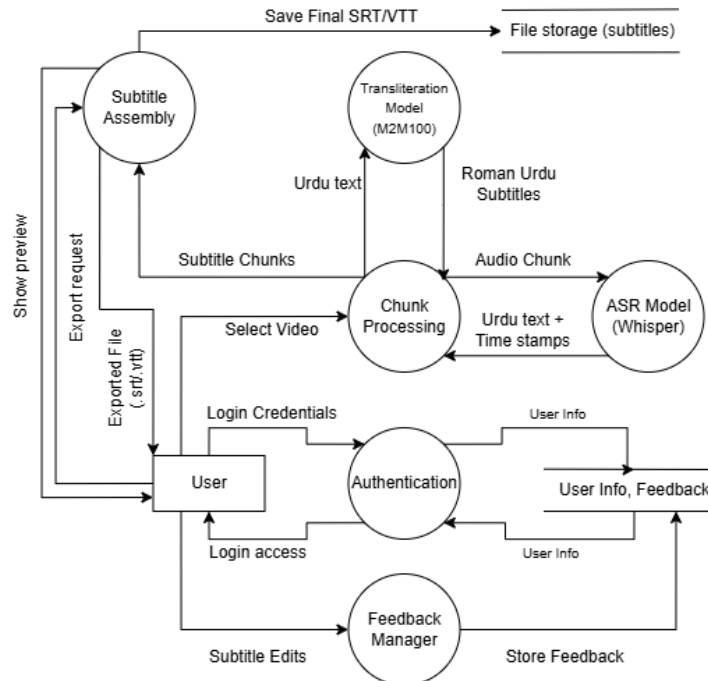
Figure 4.1.1 and Figure 4.1.2 show the main processes of the system and the data flow between them, illustrating how information moves through the different components of RomaSub.AI.

**DFD Level 0**



*Figure 4.1.1* DFD Level 0

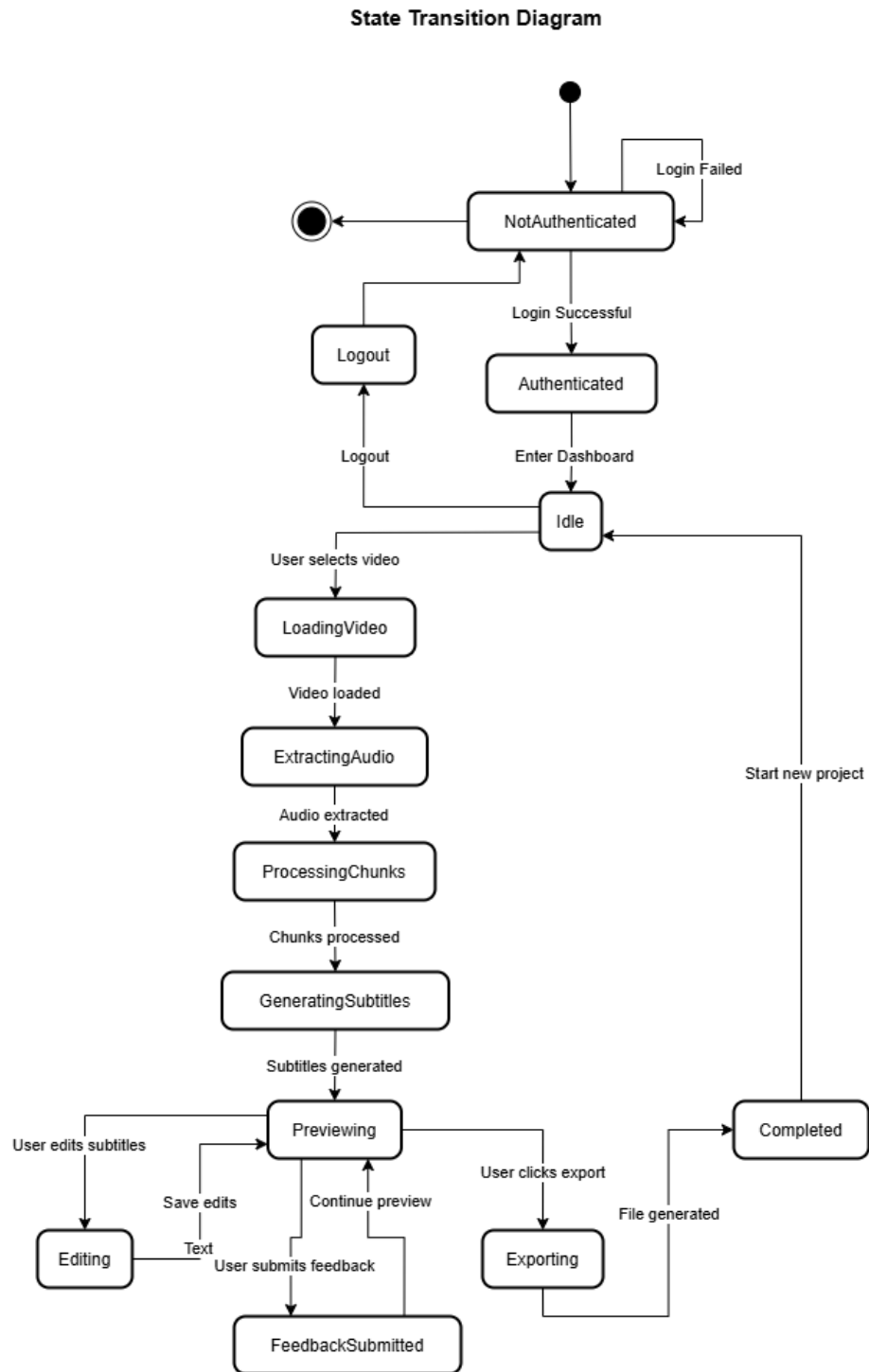
**DFD Level 1**



*Figure 4.1.2* DFD Level 1

## 4.2. State Transition Diagram:

The figure 4.2 State diagrams illustrate the lifecycle of key entities in the system.



**Figure 4.2** State Transition Diagram

## 5. Data Design:

The system uses PostgreSQL database to store user information, media metadata, feedback data, and training datasets. Files are stored in the local file system with references maintained in the database.

### 5.1 Major Data Entities & Structure

#### 1. User

**Attributes:** user\_id, email, full\_name, password\_hash, oauth\_provider, oauth\_id, is\_verified, created\_at, updated\_at

**Storage:** Relational rows in the Users table in PostgreSQL

**Purpose:** Authentication, authorization, file ownership, session tracking, and feedback association

**Notes:** No media files stored; only credentials and metadata

#### 2. Subtitle

**Attributes:** subtitle\_id, user\_id, project\_name, original\_filename, sequence\_number, start\_time, end\_time, urdu\_text, roman\_urdu\_text, confidence\_score, is\_edited, export\_format, file\_duration, created\_at, updated\_at

**Storage:** Stored in Subtitles table with a foreign key linking to Users

**Purpose:** Represents each subtitle segment extracted from uploaded media files

**Notes:** Multiple subtitles belong to a single project/file (grouped via project\_name)

#### 3. Password Reset

**Attributes:** reset\_id, user\_id, reset\_token, otp\_code, expires\_at, created\_at

**Storage:** Stored in PasswordResets table

**Purpose:** Handles OTP-based reset requests

**Notes:** Tokens automatically expire

#### 4. User Feedback

**Attributes:** feedback\_id, user\_id, subtitle\_id, feedback\_type, original\_text, corrected\_text, rating, comments, created\_at

**Storage:** Stored in **UserFeedback** table; linked to both Users and Subtitles

**Purpose:** Collects user corrections that will be used to improve AI models

**Notes:** Works as a training dataset for future model fine-tuning

## 5.2 Database / Storage Solutions

RomaSub.AI uses a hybrid storage architecture optimized for desktop-based AI workloads:

### PostgreSQL

Stores all structured data:

- Users
- Subtitles (text + metadata)
- Password resets
- User feedback

Supports indexing for fast queries across large subtitle datasets.

### Local File System

Stores:

- Original audio/video files
- Extracted audio chunks
- Exported subtitle files (SRT/VTT/TXT)

No file paths are stored in the DB; filenames are referenced through the Subtitles table.

### Model Storage

- Whisper ASR model
- Fine-tuned M2M100 transliteration model

Models are loaded into memory during processing.

### Temporary Storage

Used for:

- Intermediate ASR outputs

- Temporary files  
Automatically cleaned after completion.

## 5.3 Data Dictionary (Detailed Tables)

The following tables list all system entities in alphabetical order, with their attributes, field types, and descriptions.

### Entity: User

*Table 0.1* DD-User

| Attribute      | Type         | Description                                 |
|----------------|--------------|---------------------------------------------|
| user_id        | UUID         | Unique identifier for user                  |
| email          | VARCHAR(255) | User email address                          |
| full_name      | VARCHAR(255) | User full name                              |
| password_hash  | VARCHAR(255) | Hashed password for authentication          |
| oauth_provider | VARCHAR(50)  | Authentication method (local, google)       |
| oauth_id       | VARCHAR(255) | OAuth provider user ID                      |
| is_verified    | BOOLEAN      | Whether email is verified                   |
| created_at     | TIMESTAMP    | Time when account was created               |
| updated_at     | TIMESTAMP    | Time when profile details were last updated |

### Entity: Subtitle

*Table 0.2* DD-Subtitle

| Attribute         | Type         | Description                            |
|-------------------|--------------|----------------------------------------|
| subtitle_id       | UUID         | Unique identifier for subtitle segment |
| user_id           | UUID (FK)    | Reference to subtitle owner            |
| project_name      | VARCHAR(255) | Name of project grouping subtitles     |
| original_filename | VARCHAR(255) | Original filename of uploaded media    |

|                  |             |                                 |
|------------------|-------------|---------------------------------|
| sequence_number  | INTEGER     | Subtitle order within file      |
| start_time       | FLOAT       | Start time (seconds)            |
| end_time         | FLOAT       | End time (seconds)              |
| urdu_text        | TEXT        | Urdu transcription from ASR     |
| roman_urdu_text  | TEXT        | Roman Urdu transliteration      |
| confidence_score | FLOAT       | Confidence score for output     |
| is_edited        | BOOLEAN     | Indicates if user modified text |
| export_format    | VARCHAR(20) | Export type (SRT, VTT, TXT)     |
| file_duration    | FLOAT       | Duration of entire media file   |
| created_at       | TIMESTAMP   | Time of subtitle creation       |
| updated_at       | TIMESTAMP   | Time of last update             |

#### Entity: PasswordReset

*Table 0.3 DD-PasswordReset*

| Attribute   | Type         | Description                        |
|-------------|--------------|------------------------------------|
| reset_id    | UUID         | Unique identifier for reset entry  |
| user_id     | UUID (FK)    | Reference to requesting user       |
| reset_token | VARCHAR(255) | Secure token for password reset    |
| otp_code    | VARCHAR(6)   | One-time password for verification |
| expires_at  | TIMESTAMP    | Expiry time of reset token         |
| created_at  | TIMESTAMP    | Time when reset was initiated      |



## Entity: UserFeedback

*Table 0.4* DD-UserFeedback

| Attribute      | Type      | Description                           |
|----------------|-----------|---------------------------------------|
| feedback_id    | UUID      | Unique identifier for feedback        |
| user_id        | UUID (FK) | Reference to submitting user          |
| subtitle_id    | UUID (FK) | Reference to subtitle being corrected |
| feedback_type  | ENUM      | Type (correction, suggestion)         |
| original_text  | TEXT      | Generated Roman Urdu text             |
| corrected_text | TEXT      | User-corrected Roman Urdu text        |
| rating         | INTEGER   | User rating of subtitle quality       |
| comments       | TEXT      | Optional comments                     |
| created_at     | TIMESTAMP | Submission timestamp                  |

## 6. Algorithm and Implementation:

This section describes the core algorithms used in RomaSub.AI and explains how major system components operate at a procedural level. The system follows an object-oriented design; therefore, the implementation is described through the primary object behaviours and their member functions, represented in pseudo-code (PDL).

### 6.1 ASR Processing Algorithm

**Function:** transcribe\_audio(audio\_file)

**Input:** Audio file path

**Output:** List of timestamped text segments

**Algorithm:**

1. Load Whisper model with Urdu language setting
2. Load audio file and convert to required format

3. Process audio through Whisper model
4. Extract text segments with start and end timestamps
5. Return list of transcription segments

## 6.2 Transliteration Algorithm

**Function:** transliterate\_to\_roman(urdu\_text)

**Input:** Urdu text string

**Output:** Roman Urdu text string

**Algorithm:**

1. Load fine-tuned M2M100 model
2. Tokenize input Urdu text
3. Pass tokens through transformer model
4. Decode output tokens to Roman Urdu
5. Apply post-processing for consistency
6. Return Roman Urdu text

## 6.3 Subtitle Export Algorithm

**Function:** export\_subtitles(subtitles, format)

**Input:** List of subtitle segments, export format (SRT/VTT)

**Output:** Formatted subtitle file

**Algorithm:**

1. For each subtitle segment
2. Format timestamp according to chosen format
3. Add sequence number (for SRT)
4. Write text content
5. Save to file with appropriate extension

## 7. Software Requirements Traceability Matrix:

The following table maps functional requirements to design components:

*Table 0.1* Traceability Matrix

| Req. No. | Ref. Item         | Design Component | Component Items                                |
|----------|-------------------|------------------|------------------------------------------------|
| FR01     | Data Flow Diagram | User             | register(), login(), authenticate()            |
| FR02     | Data Flow Diagram | MediaHandler     | uploadFile(), extractAudio(), validateFormat() |

| Req. No. | Ref. Item         | Design Component | Component Items                                     |
|----------|-------------------|------------------|-----------------------------------------------------|
| FR03     | Data Flow Diagram | ASRService       | transcribe(), processAudio(), getTimestamps()       |
| FR04     | Data Flow Diagram | DatasetGenerator | collectSamples(), alignData(), exportDataset()      |
| FR05     | Data Flow Diagram | ModelTrainer     | fineTune(), validate(), saveModel()                 |
| FR06     | Data Flow Diagram | Transliterator   | convert(), postProcess(), getConfidence()           |
| FR07     | Data Flow Diagram | SubtitleDisplay  | syncWithVideo(), render(), updatePosition()         |
| FR08     | Data Flow Diagram | SubtitleEditor   | editText(), adjustTiming(), preview()               |
| FR09     | Data Flow Diagram | ExportManager    | formatSRT(), formatVTT(), saveFile()                |
| FR10     | Data Flow Diagram | FeedbackSystem   | collectFeedback(), processFeedback(), updateModel() |

## 8. Interface Design:

The user interface is designed as a Flutter desktop application with a clean and simple layout. Users can easily navigate through different sections using the sidebar menu.

### 8.1. Screen Images:

These screens represent the core user interface of the RomaSub.AI system, showing how users interact with key features such as authentication, journey recording, map navigation, social sharing, and profile management. The design ensures a consistent and intuitive experience across both web and mobile platforms. Each interface focuses on clarity, ease of use, and smooth access to primary system functions.



### Welcome to RomaSub.AI

☐ I agree to the Terms & Conditions

Create account

Already have an account? [Log in](#)

**Figure 0.1** Register Page



### Login to RomaSub.AI

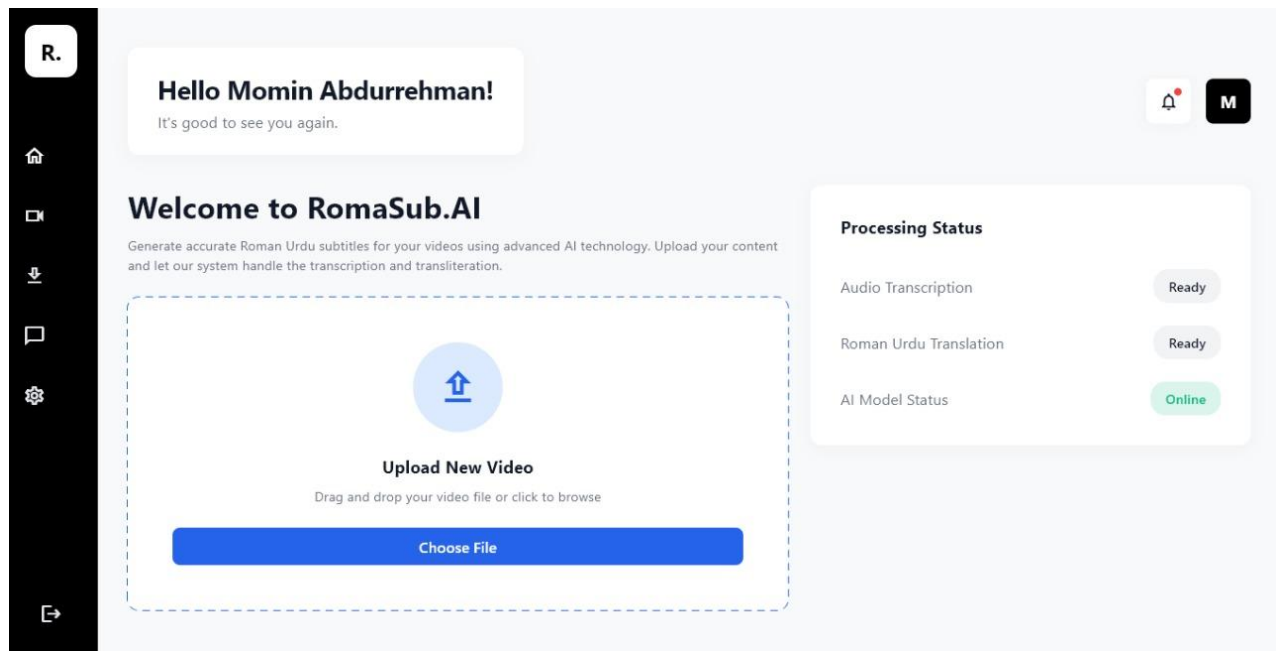
[Forgot Password?](#)

Continue

Don't have an account? [Sign Up](#)

[Terms of use](#) | [Privacy policy](#)

**Figure 0.2** Login Page



*Figure 0.3* Home Page / Dashboard

## 8.2. Screen Objects and Actions:

| Screen    | Object          | Action                                |
|-----------|-----------------|---------------------------------------|
| Login     | Email Field     | Enter user email                      |
| Login     | Password Field  | Enter user password                   |
| Login     | Sign In Button  | Authenticate user and go to Dashboard |
| Dashboard | Upload Area     | Upload audio/video file               |
| Dashboard | Project Card    | View project status and options       |
| Editor    | Video Player    | Play/pause video with subtitles       |
| Editor    | Text Editor     | Edit subtitle text and timing         |
| Export    | Format Dropdown | Select export format (SRT/VTT)        |
| Export    | Download Button | Download subtitle file                |

## 9 Appendix I

- **Agile Modeling Guide**

<https://agilemodeling.com/essays/bestpractices.htm>

- **UML Reference**

<https://staruml.io/>

<https://www.diagrams.net/>

- **Flutter Documentation**

<https://docs.flutter.dev/>

<https://pub.dev/>